SRM TRP ENGINEERING COLLEGE
(Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai)
NH 45, Tirchy – Chennai Highway, Irungalur, Mannachanallur Taluk, Tiruchirappalli
District, Tamil Nadu 621105

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

MINOR PROJECT REPORT

HEALTHCARE DATA ANALYSIS AND DISEASE RISK PREDICTION

Prediction Problem Definition, Model Development, Evaluation and Comparison

Submitted by

SHALINI R
SAKTHIS SREE S
II AI & DS – C

Under the Guidance of

V. EZHIL SWANLY

Submission Date: 24 October 2026

CERTIFICATE

This is to certify that the mini project report titled “Healthcare Data Analysis and Disease Risk Prediction” is submitted by SHALINI R and SAKTHIS SREE S of the Department of Artificial Intelligence and Data Science, SRM TRP Engineering College, Tiruchirappalli. The report presents a machine-learning workflow covering problem definition, data preparation, model development, evaluation and comparison.

ACKNOWLEDGEMENT

We express our sincere gratitude to **SRM TRP Engineering College** and the **Department of Artificial Intelligence and Data Science** for providing us with the opportunity, infrastructure, and facilities to successfully complete this mini project.

We express our heartfelt gratitude to our **Dr.Perumalraja, Professor & Head, Department of Artificial Intelligence and Data Science,** for providing constant encouragement, valuable support, and a motivating academic environment that greatly contributed to the successful completion of this project.

We sincerely thank our project guide, **Mr. V. Ezhil Swanly, Assistant Professor, Department of Artificial Intelligence and Data Science**, for his valuable guidance, continuous encouragement, constructive suggestions, and unwavering support throughout the development of this project.

We also extend our sincere thanks to all the faculty members of the Department of Artificial Intelligence and Data Science for their valuable inputs and encouragement. We are grateful to our classmates and family members for their continuous support, cooperation, and motivation during the preparation of the healthcare dataset, data analysis notebook, and project report.

Finally, we acknowledge the healthcare datasets and spreadsheets used for this study, as well as the open-source Python libraries and tools that supported the data preprocessing, analysis, visualization, and report preparation. Their availability greatly contributed to the successful completion of this mini project.

SHALINI R

SAKTHIS SREE S

TABLE OF CONTENTS

- Abstract
- Chapter 1 – Problem Definition
- Chapter 2 – Data Preparation
- Chapter 3 – Model Development
- Chapter 4 – Model Evaluation
- Chapter 5 – Model Comparison
- Chapter 6 – Result Analysis
- Conclusion
- References
- Appendix – Code and Output

ABSTRACT

This project explores a supervised machine-learning workflow for classifying the Disease category from healthcare indicators. The supplied project report describes 15 patient records and eight columns: Age, Gender, BMI, Smoking, Glucose, BP, Cholesterol and Disease. The raw workbook was reported to contain missing values in Age, BMI, Cholesterol and Disease, while the cleaned workbook had no missing values in the inspected fields. The earlier exploratory analysis reported positive sample correlations between Glucose and BP (0.85), BMI and Glucose (0.78), and BMI and BP (0.72).

Three classifiers are included: Logistic Regression, Decision Tree and Random Forest. The notebook calculates accuracy, macro precision, macro recall, macro F1-score, classification reports, confusion matrices and feature importance. Actual model outputs must be generated by running the code with the real Excel workbook. Since those run outputs were not supplied in this chat, output placeholders are included rather than invented scores. The dataset is very small and is suitable only for an educational demonstration, not clinical diagnosis.

CHAPTER 1 – PROBLEM DEFINITION

1.1 Prediction Problem

The proposed task is supervised classification: use patient information to predict the encoded Disease category.

1.2 Target Variable

Target variable: Disease. The numeric category codes must be mapped to disease names using a verified data dictionary. If the mapping is unavailable, refer to them as encoded categories.

1.3 Feature Variables

Feature	Description	Role
Age	Patient age in years	Input feature
Gender	Encoded gender category	Categorical input
BMI	Body mass index	Input feature
Smoking	Encoded smoking status	Categorical input
Glucose	Glucose measurement	Input feature
BP	Blood pressure measurement	Input feature
Cholesterol	Cholesterol measurement	Input feature
Disease	Encoded disease category	Target variable

1.4 Objectives

- Inspect the raw and cleaned datasets.
- Prepare the input features and target.
- Train three classification models.
- Evaluate and compare model performance.
- Discuss feature importance, limitations and future improvements.

CHAPTER 2 – DATA PREPARATION

2.1 Dataset Overview

Item	Known details from the supplied project report
Raw dataset	15 records, 8 columns
Cleaned dataset	15 records, 8 columns
Raw missing values	Age: 2; BMI: 2; Cholesterol: 1; Disease: 2
Cleaned missing values	0 in the eight inspected columns
Columns	Age, Gender, BMI, Smoking, Glucose, BP, Cholesterol, Disease

2.2 Preparation Procedure

- Upload the cleaned Excel workbook in Google Colab.
- Inspect dimensions, column names and missing values.
- Check required columns and convert feature fields to numeric.
- Remove records without a target label; impute missing input values within each model pipeline.
- Separate X (seven features) and y (Disease).
- Split the dataset into training and test sets. With only 15 records, the split and resulting metrics are unstable.

2.3 Data Preparation Output

Paste the actual output printed by Colab for dataset shape, column names, missing values, disease category counts and training/testing record counts.

```
Libraries imported successfully!

Upload your cleaned healthcare Excel file:
Choose Files cleaned_he...data (1).xlsx
cleaned_healthcare_data (1).xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 4394 bytes, last modified: 10/28/2026 - 100% done
Saving cleaned_healthcare_data (1).xlsx to cleaned_healthcare_data (1) (1).xlsx

Dataset loaded successfully!

First 5 records:
   Age  Gender   BMI  Smoking  Glucose   BP  Cholesterol  Disease
0   52       1  28.5        1      130  140        220.0        1
1   61       0  32.1        0      210  150        260.0        2
2   48       0  24.3        1       98  120        190.0        3
3   45       1  27.8        0      110  130        210.0        1
4   58       0  27.8        1      130  135        240.0        2

Dataset Shape: (15, 8)

Column Names: ['Age', 'Gender', 'BMI', 'Smoking', 'Glucose', 'BP', 'Cholesterol', 'Disease']

Missing Values:
Age            0
Gender         0
BMI            0
Smoking        0
Glucose        0
BP             0
Cholesterol    0
Disease        0
dtype: int64
```

```
Prepared Dataset Shape: (15, 8)

Disease Category Counts:
Disease
1    5
2    4
3    4
0    2
Name: count, dtype: int64
Training records: 11
Testing records: 4
Training categories: 4
Testing categories: 4

==================================================
```

CHAPTER 3 – MODEL DEVELOPMENT

3.1 Logistic Regression

A baseline classifier that estimates class probabilities from a weighted combination of input features. The pipeline imputes missing values and standardizes features.

3.2 Decision Tree

A tree-based classifier that divides records using feature-based rules. It is interpretable but may overfit small datasets.

3.3 Random Forest

An ensemble classifier that combines multiple decision trees. It can model nonlinear patterns and provides feature-importance values, but results are unstable on a tiny dataset.

3.4 Model Training Code and Output

The complete runnable program is provided in the Appendix. After running it, insert the output confirming the three model names and their evaluation reports.

CHAPTER 4 – MODEL EVALUATION

Metric	Meaning
Accuracy	Fraction of test predictions that are correct
Precision	How many predictions for a class are correct
Recall	How many actual examples of a class are identified
F1-score	Harmonic mean of precision and recall
Confusion Matrix	Counts correct and incorrect predictions by class

4.1 Logistic Regression Evaluation

Insert the actual accuracy, precision, recall, F1-score, classification report and confusion matrix generated by Colab.

```
Logistic Regression
Accuracy: 0.5
Precision: 0.375
Recall: 0.5
F1 Score: 0.4167

Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.50      1.00      0.67         1
           2       1.00      1.00      1.00         1
           3       0.00      0.00      0.00         1

    accuracy                           0.50         4
   macro avg       0.38      0.50      0.42         4
weighted avg       0.38      0.50      0.42         4
```

4.2 Decision Tree Evaluation

Insert the actual evaluation metrics, classification report and confusion matrix.

```
Decision Tree
Accuracy: 0.5
Precision: 0.3333
Recall: 0.5
F1 Score: 0.375

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       0.33      1.00      0.50         1
           2       0.00      0.00      0.00         1
           3       0.00      0.00      0.00         1

    accuracy                           0.50         4
   macro avg       0.33      0.50      0.38         4
weighted avg       0.33      0.50      0.38         4
```

4.3 Random Forest Evaluation

Insert the actual evaluation metrics, classification report and confusion matrix.

```
Random Forest
Accuracy: 0.25
Precision: 0.0833
Recall: 0.25
F1 Score: 0.125

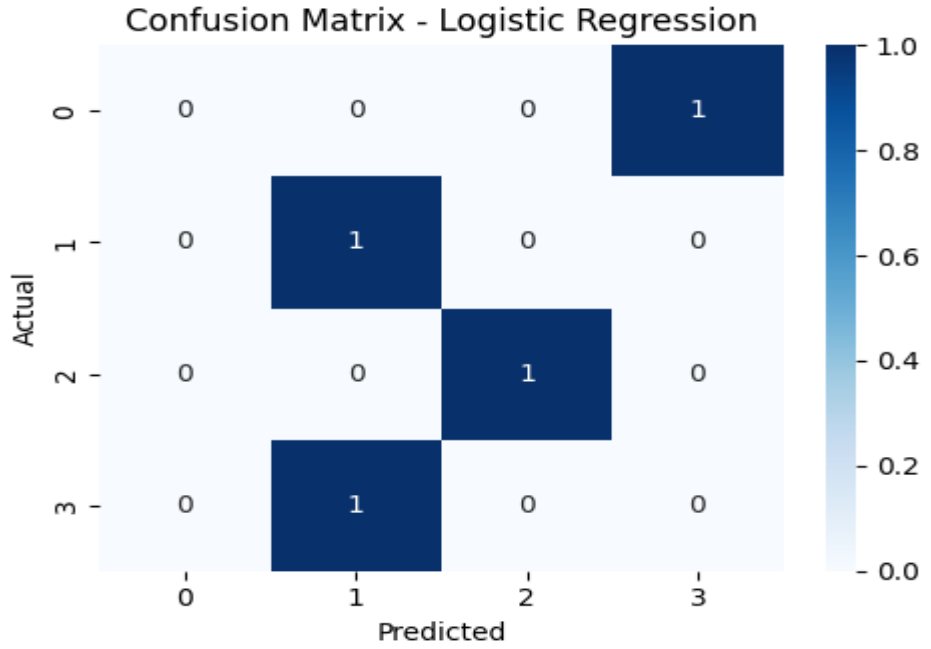
Classification Report:
              precision    recall  f1-score   support

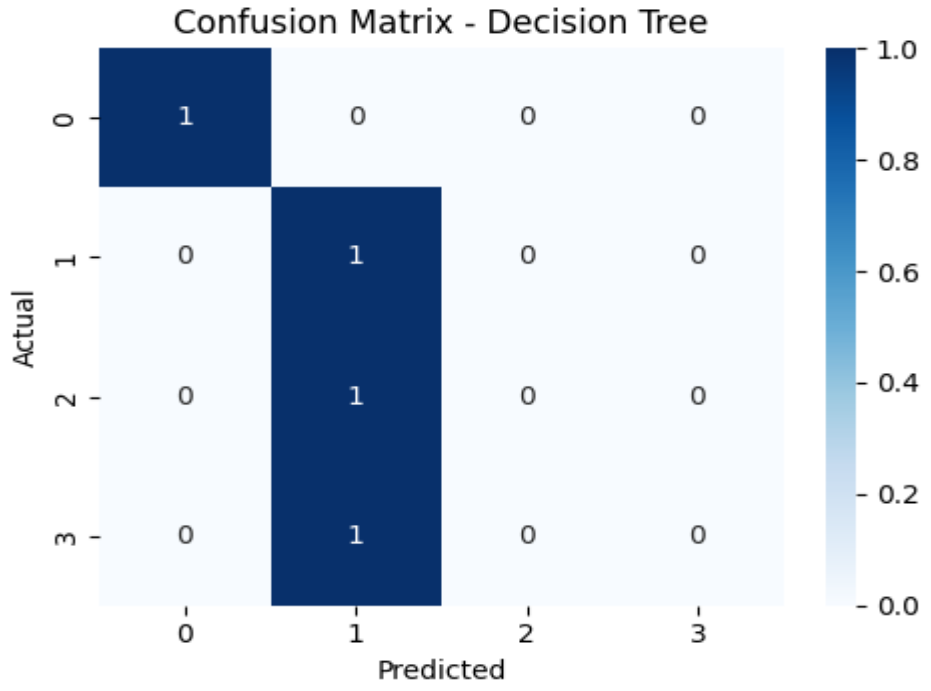
           0       0.00      0.00      0.00         1
           1       0.33      1.00      0.50         1
           2       0.00      0.00      0.00         1
           3       0.00      0.00      0.00         1

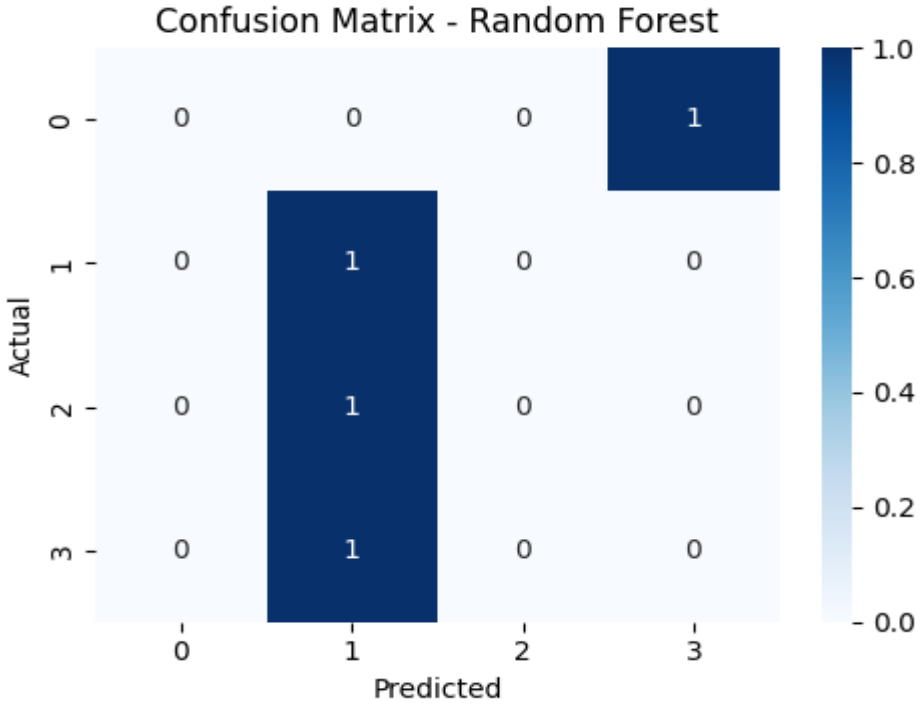
    accuracy                           0.25         4
   macro avg       0.08      0.25      0.12         4
weighted avg       0.08      0.25      0.12         4
```

Note: Do not type guessed scores. A very small test set means a single prediction can change the metrics considerably.

4.4 Confusion Matrices







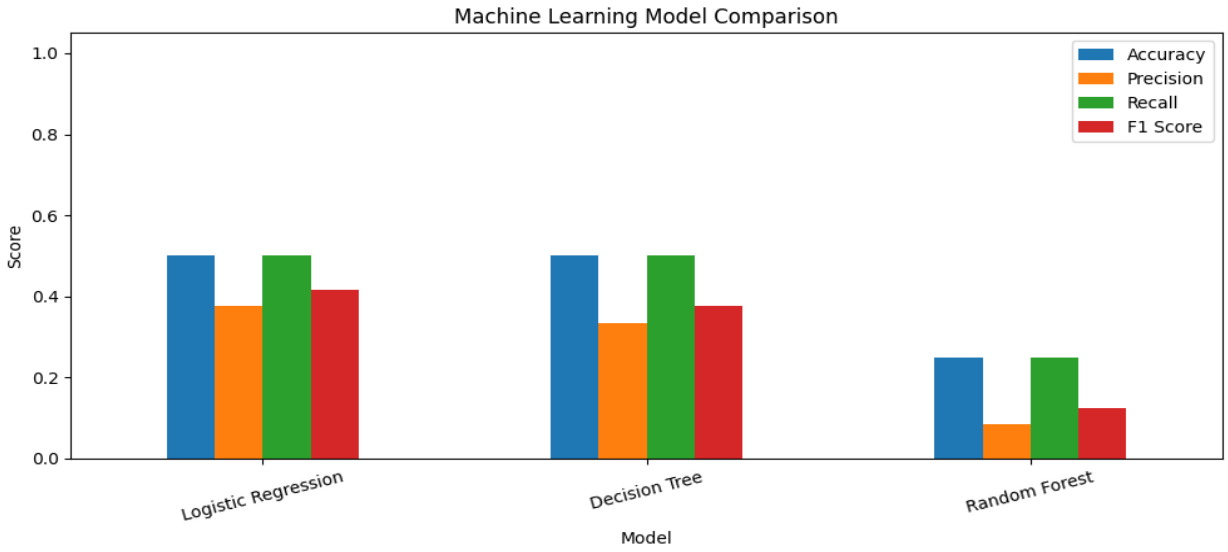
CHAPTER 5 – MODEL COMPARISON

The program sorts the three models by macro F1-score. Use the printed comparison table to identify the strongest model on this particular test split, and consider precision, recall and confusion matrices as well.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Logistic Regression	0.50	0.3750	0.50	0.4167
Decision Tree	0.50	0.3333	0.50	0.3750
Random Forest	0.25	0.0833	0.25	0.1250

```
MODEL COMPARISON
                Model  Accuracy  Precision  Recall  F1 Score
Logistic Regression      0.50     0.3750    0.50    0.4167
      Decision Tree      0.50     0.3333    0.50    0.3750
      Random Forest      0.25     0.0833    0.25    0.1250

Best model on this test split: Logistic Regression
Based on macro F1 score.
This is a preliminary result, not clinical validation.
```



The best model is the one with the most appropriate test performance on the observed run. This does not mean it is clinically validated or ready for real-world use.

CHAPTER 6 – RESULT ANALYSIS

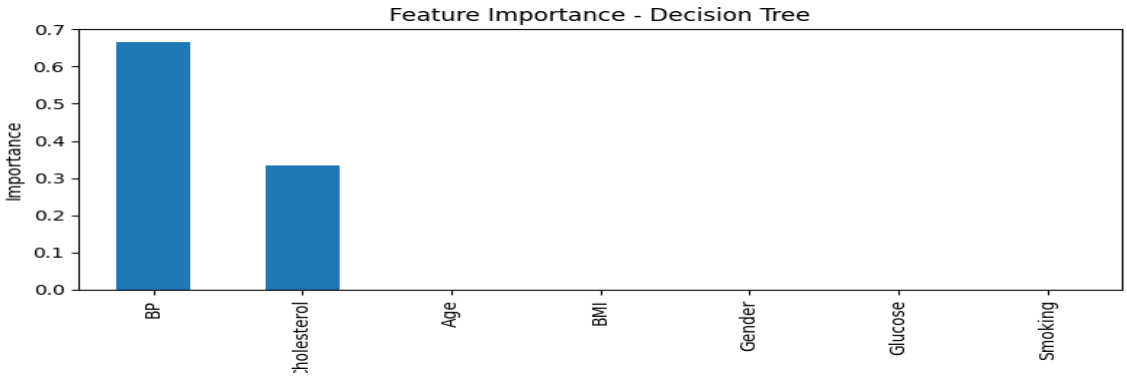
6.1 Existing Exploratory Findings

- The supplied dataset contains 15 records and eight variables.
- The raw spreadsheet has missing values; the cleaned spreadsheet reportedly has none in the inspected fields.
- The reported age distribution spans approximately 35–65 years.
- The earlier correlation matrix reported Glucose–BP = 0.85, BMI–Glucose = 0.78 and BMI–BP = 0.72.
- These sample correlations do not establish causation.

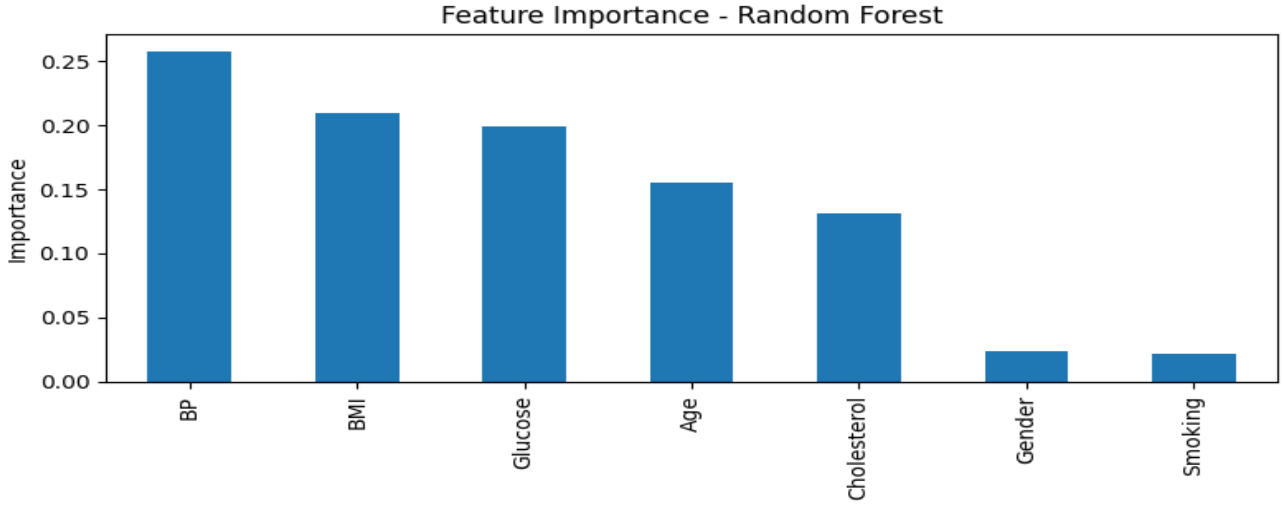
6.2 Feature Importance

The code prints feature-importance values and graphs for Decision Tree and Random Forest. Insert the actual results below. Feature importance describes how a trained model uses features; it does not prove causation.

```
Feature Importance - Decision Tree
BP             0.666667
Cholesterol    0.333333
Age            0.000000
BMI            0.000000
Gender         0.000000
Glucose        0.000000
Smoking        0.000000
dtype: float64
```




```
Feature Importance - Random Forest
BP              0.257700
BMI             0.209715
Glucose         0.199312
Age             0.155590
Cholesterol     0.131702
Gender          0.024006
Smoking         0.021976
dtype: float64
```




```
PROJECT COMPLETED
Dataset records used: 15
Input features: ['Age', 'Gender', 'BMI', 'Smoking', 'Glucose', 'BP', 'Cholesterol']
Target variable: Disease
Models trained: 3

Final Comparison:
                Model  Accuracy  Precision  Recall  F1 Score
Logistic Regression      0.50     0.3750    0.50    0.4167
      Decision Tree      0.50     0.3333    0.50    0.3750
      Random Forest      0.25     0.0833    0.25    0.1250

NOTE: The dataset contains only a small number of records. The metrics may be unstable and must not be used for medical diagnosis or treatment decisions.
```

6.3 Limitations

- Only 15 records are available, so results cannot be generalized reliably.
- The numeric category codes need a documented data dictionary.
- The test set is very small, making evaluation metrics unstable.
- The project is educational and must not be used for diagnosis or treatment.

6.4 Future Improvements

- Collect a larger, representative and ethically sourced dataset.
- Verify labels, units, duplicates and measurement ranges.
- Use suitable cross-validation when enough examples per class are available.
- Evaluate class-specific performance and consult qualified healthcare professionals before any real-world use.

6.5 Real-World Applications

Validated healthcare classification research may help analyze patient populations or prioritize records for professional review. This student project is not a validated medical system.

CONCLUSION

This report defines Disease as the target and identifies seven input features. It describes preparation and three classifiers—Logistic Regression, Decision Tree and Random Forest—and includes evaluation and comparison procedures. Actual scores and graphs must be added from a successful Colab run. The small dataset and undocumented category mappings limit interpretation, and the results must not be used for clinical decisions.

REFERENCES

- Supplied project report: Healthcare Data Analysis and Disease Risk Prediction.
- Pandas documentation: https://pandas.pydata.org/docs/
- Matplotlib documentation: https://matplotlib.org/stable/
- Seaborn documentation: https://seaborn.pydata.org/
- Scikit-learn documentation: https://scikit-learn.org/stable/

APPENDIX – COMPLETE CODE

```
# HEALTHCARE DATA ANALYSIS AND DISEASE RISK PREDICTION
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)print("Libraries imported successfully!")

# 2. Upload Excel Dataset
print("\nUpload your cleaned healthcare Excel file:")
uploaded = files.upload()
file_path = next(iter(uploaded.keys()))
df = pd.read_excel(file_path)
# 3. Clean Column Names
df.columns = df.columns.astype(str).str.strip()
print("\nDataset loaded successfully!")
print("\nFirst 5 records:")
print(df.head())
print("\nDataset Shape:", df.shape)
print("\nColumn Names:", df.columns.tolist())
print("\nMissing Values:")
print(df.isnull().sum())
# 4. Validate Required Columns
target = "Disease"
features = [
    "Age", "Gender", "BMI", "Smoking",
    "Glucose", "BP", "Cholesterol"
]
required = features + [target]
missing_columns = [c for c in required if c not in df.columns]
if missing_columns:
    raise ValueError(
        f"Missing columns: {missing_columns}. "
        f"Available columns: {df.columns.tolist()}"
    )

# 5. Data Preparation
df = df.dropna(subset=[target]).copy()

# Convert feature columns to numeric
for col in features:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Remove rows where all feature values are missing
df = df.dropna(subset=features, how="all")
X = df[features]
y = df[target]
```

```
print("\nPrepared Dataset Shape:", df.shape)
print("\nDisease Category Counts:")
print(y.value_counts())
if y.nunique() < 2:
    raise ValueError(
        "At least two Disease categories are needed for classification."
    )
# 6. Train-Test Split (Fixed)
class_counts = y.value_counts()
number_of_classes = y.nunique()

# Ensure the test set can hold at least one record per class
test_records = max(
    int(np.ceil(0.20 * len(y))),
    number_of_classes
)

# Ensure enough records remain for training
if test_records >= len(y):
    raise ValueError(
        "Dataset is too small for a train-test split. "
        "Add more records to the dataset."
    )
stratify_y = (
    y if class_counts.min() >= 2 else None
)
X_train, X_test, y_train, y_test = train_test_split(
    X,
   y,
    test_size=test_records,
    random_state=42,
    stratify=stratify_y
)
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
print("Training categories:", y_train.nunique())
print("Testing categories:", y_test.nunique())

# 7. Create Three Machine Learning Models
models = {
    "Logistic Regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(
            max_iter=2000, class_weight="balanced"
        ))
    ]),
```

```
    "Decision Tree": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", DecisionTreeClassifier(
            max_depth=3,
            random_state=42,
            class_weight="balanced"
        ))
    ]),
    "Random Forest": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", RandomForestClassifier(
            n_estimators=100,
            max_depth=3,
            random_state=42,
            class_weight="balanced"
        ))
    ])
}

# 8. Train and Evaluate Models
results = []
trained_models = {}
for name, model in models.items():
    print("\n" + "=" * 50)
    print(name)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    trained_models[name] = model
    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(
        y_test, predictions,
        average="macro", zero_division=0
    )
    recall = recall_score(
        y_test, predictions,
        average="macro", zero_division=0
    )
    f1 = f1_score(
        y_test, predictions,
        average="macro", zero_division=0
    )
    results.append({
        "Model": name,
        "Accuracy": round(accuracy, 4),
        "Precision": round(precision, 4),
        "Recall": round(recall, 4),
        "F1 Score": round(f1, 4)
    })
```

```
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1 Score:", round(f1, 4))
    print("\nClassification Report:")
    print(classification_report(
        y_test, predictions, zero_division=0
    ))
    # Confusion Matrix
    cm = confusion_matrix(
        y_test, predictions, labels=np.unique(y)
    )
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"Confusion Matrix - {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()

# 9. Compare All Models
comparison = pd.DataFrame(results)
comparison = comparison.sort_values(
    by="F1 Score", ascending=False
)
print("\nMODEL COMPARISON")
print(comparison.to_string(index=False))

# 10. Display Best Model on This Test Split
best_model_name = comparison.iloc[0]["Model"]

print("\nBest model on this test split:", best_model_name)
print("Based on macro F1 score.")
print("This is a preliminary result, not clinical validation.")

# 11. Model Comparison Graph
comparison.plot(
    x="Model",
    y=["Accuracy", "Precision", "Recall", "F1 Score"],
    kind="bar",
    figsize=(10, 5)
)
plt.title("Machine Learning Model Comparison")
plt.ylabel("Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

```
# 12. Feature Importance (Decision Tree and Random Forest)
for name in ["Decision Tree", "Random Forest"]:
    fitted_pipeline = trained_models[name]
    classifier = fitted_pipeline.named_steps["model"]
    importance = pd.Series(
        classifier.feature_importances_,
        index=features
    ).sort_values(ascending=False)

    print(f"\nFeature Importance - {name}")
    print(importance)

    importance.plot(
        kind="bar",
        figsize=(8, 4),
        title=f"Feature Importance - {name}"
    )
    plt.ylabel("Importance")
    plt.tight_layout()
    plt.show()

# 13. Final Summary
print("\nPROJECT COMPLETED")
print("Dataset records used:", len(df))
print("Input features:", features)
print("Target variable:", target)
print("Models trained:", len(trained_models))
print("\nFinal Comparison:")
print(comparison.to_string(index=False))

print(
    "\nNOTE: The dataset contains only a small number of records. "
    "The metrics may be unstable and must not be used for medical "
    "diagnosis or treatment decisions."
)
```